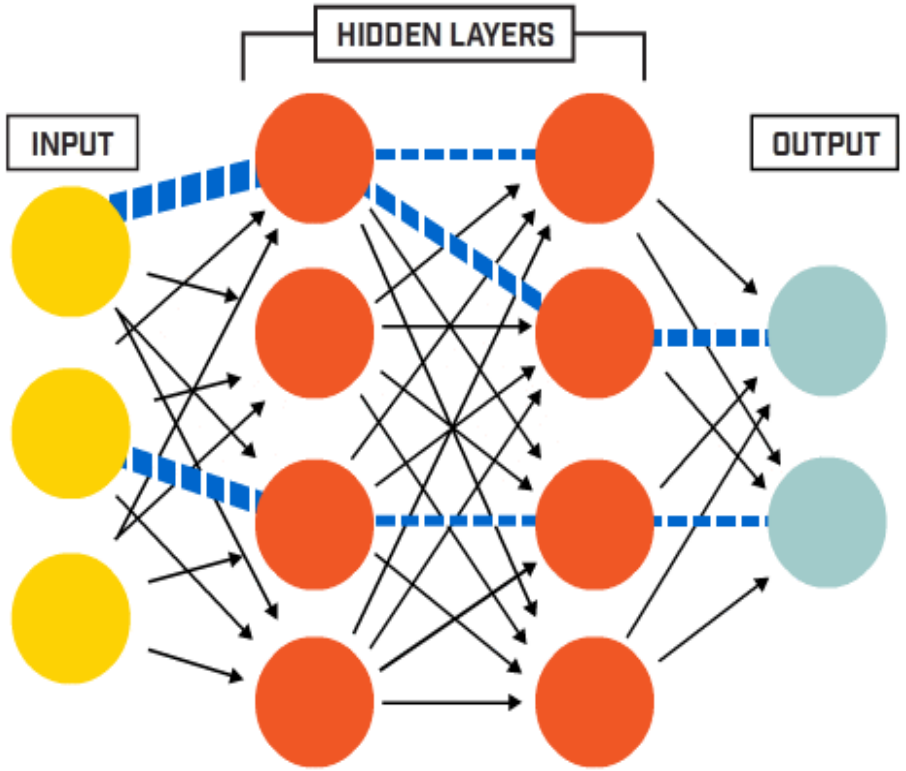
RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

ARTIFICIAL NEURAL NETWORK AND DEEP LEARNING



Dr. Viswavardhan Reddy Karna
Dept of AIML
RVCE

Course Contents

Semester: V

ARTIFICIAL NEURAL NETWORKS AND DEEP LEARNING

Category: Professional Core Course

(Theory and Practice)

Course Code	**:**	**AI253IA**	**CIE**	**:**	**100+50 Marks**
Credits: L: T: P	**:**	**3:0:1**	**SEE**	**:**	**100 + 50 Marks**
Total Hours	**:**	**45L+30P**	**SEE Duration**	**:**	**3.00 + 3.00 Hours**

Unit-I — 9Hrs.

Neural Networks: Introduction to NN, models of neuron and network architectures. **Learning Processes**: Different types of learning processes, Learning with and without teacher, Memory, statistical learning theory.

Single layer perceptron: Adaptive filter problem, least mean square algorithm, learning rate, Learning rate annealing techniques, perceptron and perceptron convergence theorem.

Multilayer Perceptron: Back propagation algorithm, Sequential and batch modes of training, stopping criteria, XOR problem, and some numerical problems

Unit – II — 9Hrs.

Convolutional Neural Networks: Introduction, Historical Perspective and Biological Inspiration. **Basic Structure of a Convolutional Network:** Padding, Strides, Typical Settings, The ReLU Layer, Pooling, Fully Connected Layers, The Interleaving Between Layers, Local Response Normalization, Hierarchical Feature Engineering.

Training a Convolutional Network: Back propagating Through Convolutions, Back propagation as Convolution with Inverted/Transposed Filter, Convolution/Back propagation as Matrix Multiplications, Data Augmentation.

Applications of CNN: Content based image retrieval, Object Localization, Object Detection, Natural Language and sequence learning, and Video classification

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Course Contents

Unit –III	9Hrs.
Recurrent Neural Networks: Introduction and expressiveness of RNN. **Basic Structure of a RNN:** Language Modeling Example of RNN, Generating a Language Sample, Back propagation Through Time, Bidirectional Recurrent Networks, Multilayer Recurrent Networks. **Echo-State Networks, Long Short-Term Memory (LSTM), Gated Recurrent Units (GRUs)** **Applications of Recurrent Neural Networks:** Automatic Image Captioning, Sequence-to-Sequence Learning and Machine Translation, Sentence-Level Classification, Token-Level Classification with Linguistic Features, Time-Series Forecasting and Prediction, Temporal Recommender Systems, Secondary Protein Structure Prediction, End-to-End Speech Recognition, Handwriting Recognition	

Unit –IV	9Hrs.
Deep Reinforcement Learning: Introduction, **Stateless Algorithms:** Multi-Armed Bandits, Na¨ıve Algorithm, Greedy algorithm, Upper Bounding Methods **The Basic Framework of Reinforcement Learning:** Challenges of Reinforcement Learning, Simple Reinforcement Learning for Tic-Tac-Toe, ole of Deep Learning and a Straw-Man Algorithm. **Bootstrapping for Value Function Learning:** Deep Learning Models as Function Approximators, Example: Neural Network for Atari Setting, On-Policy Versus Off-Policy Methods: SARSA, Modeling States Versus State-Action Pairs, **Monte Carlo Tree Search** **Case Studies:** Alpha Go: Championship Level Play at Go, Alpha Zero: Enhancements to Zero Human Knowledge, Self-Learning Robots: Deep Learning of Locomotion Skills, Deep Learning of Visuomotor Skills, Building Conversational Systems: Deep Learning for Chat-Bots, Self-Driving Cars	

Course Contents

Unit –V	9Hrs

Advanced Topics in Deep Learning: Attention Mechanisms, Attention Mechanisms for Machine Translation, Neural Turing Machines, Competitive learning, Limitations of neural networks. Cars

Generative Adversarial Networks (GANs): Training a GAN, Comparison with variational auto encoder, Using GANs for generating Image data, conditional GANs.

Laboratory Component

Group of two students belong to the same batch are required to implement an engineering application using any one of the deep learning techniques, CNN, RNN and Reinforcement learning.

Examples:

CNN: Biometric authentication using CNN, Object identification and recognition, Emotion recognition, Auto translation, Document classification etc.

RNN: Language translation, Generating image descriptions, Speech recognition etc.

Reinforcement Learning: Real-time bidding, Recommendation systems, Traffic control systems etc.

Course Outcomes: After completing the course, the students will be able to:-	
CO1	Describe basic concepts of neural networks, its applications and various learning models
CO2	Analyze different network architectures, learning tasks, CNN, and deep learning models
CO3	Investigate and apply neural networks model and learning techniques to solve problems related to society and industry.
CO4	Demonstrate a prototype application developed using any NN tools and APIs.
CO5	Appraise the knowledge of neural networks and deep learning as an individual/as an team member.

Course Contents

Reference Books

1	Neural Networks – A Comprehensive Foundation, Simon Haykin, 2nd Edition, PHI, 2005.
2	Neural Networks and Deep learning: A Textbook ,Charu C Aggarwal, Springer International Publishing AG, ISBN 978-3-319-94462-3 ISBN 978-3-319-94463-0 (eBook), https://doi.org/10.1007/978-3-319-94463-0, 2018
3	Deep Learning (Adaptive Computation and Machine Learning Series),,Ian Good fellow, Yoshua Bengio and Aaron Courville, MIT Press ,2017, ISBN-13: 978-0262035613.
4	Fundamentals of Artificial Neural Networks ,M H Hassoun, MIT Press, 2010, ISBN-13: 978-0262514675.

Unit -1

- **Neural Networks**
  - What is NN
  - Models of neurons and network architecture

- **Learning Processes**
  - Different Types of Learning Process
  - Learning with and without a teacher
  - Learning tasks
  - Memory and adaptation
  - Statistical Learning

Unit -1

- **Single layer perceptron**
  - Adaptive filter problem
  - Least mean square algorithm, learning rate.
  - Learning rate annealing techniques, perceptron, and perceptron convergence theorem.
- **Multilayer Perceptron**
  - Back propagation algorithm
  - Sequential and batch modes of training, stopping criteria.
  - XOR problem, and some numerical problems

What is a Neural Network?

- Inspired by the human brain
- Consists of interconnected nodes (neurons)
- Learns from data to make predictions/decisions
- Applications: Image recognition, NLP, robotics, etc.

Biological Neuron vs. Artificial Neuron

Biological Neuron

- Dendrites (inputs)
- Soma (processing)
- Axon (output)

Artificial Neuron

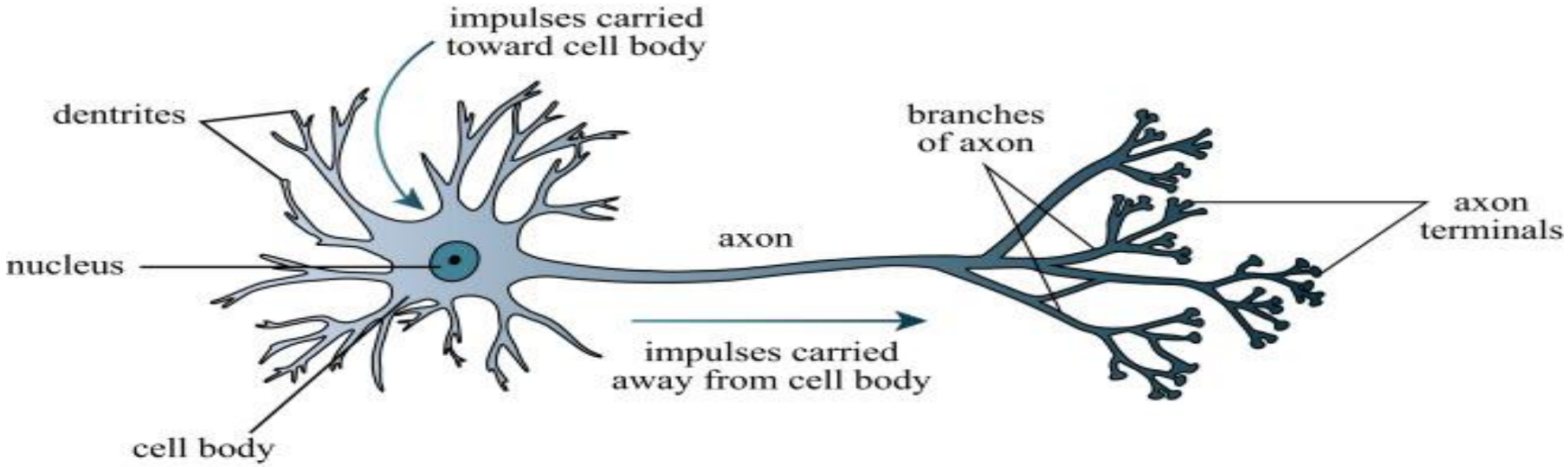
- Inputs (x1, x2, …, xn)
- Weights (w1, w2, …, wn)
- Summation + Activation Function
- Output (y)

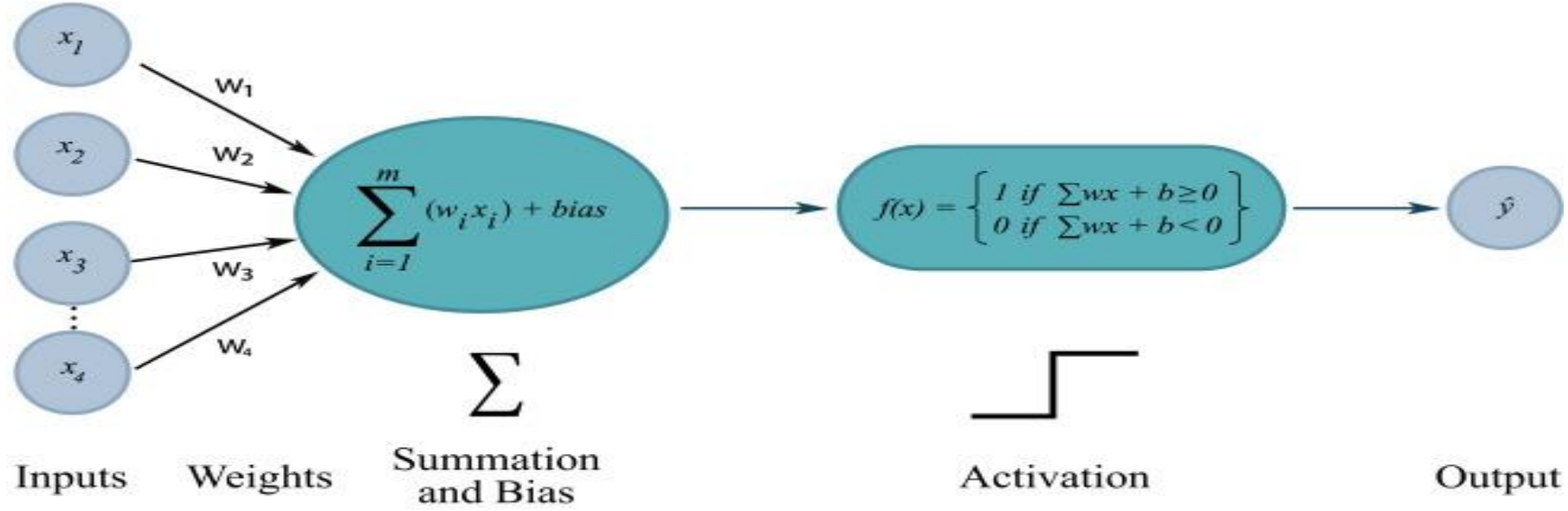
Mathematical Model of a Neuron

- Input: $x_1, x_2, \ldots, x_n$
- Weights: $w_1, w_2, \ldots, w_n$
- Net Input: $z = \sum w_i x_i + b$
- Activation Function: f(z)
- Output: $y = f(z)$

Activation Functions: Step, Sigmoid, ReLU, Tanh

Unit -1


impulses carried
toward cell body
dentrites
branches
of axon
axon
terminals
nucleus
axon
impulses carried
away from cell body
cell body


x_1
x_2
x_3
x_4
w_1
w_2
w_3
w_4
$\sum_{i=1}^{m}(w_i x_i) + bias$
$f(x) = \begin{cases} 1 \text{ if } \sum wx + b \geq 0 \\ 0 \text{ if } \sum wx + b < 0 \end{cases}$
$\hat{y}$
Inputs
Weights
Summation
and Bias
Activation
Output

Unit -1

- **Biological Neuron**
- **Dendrites** → Receive input signals from other neurons.
- **Cell body (soma)** → Processes signals.
- **Nucleus** → Controls the neuron.
- **Axon** → Carries the processed signal away.
- **Axon terminals** → Pass the signal to the next neuron.
- So: **Input → Processing → Output.**

Artificial Neuron: In ANN, we mimic the above structure with math:

•**Inputs (x_1, x_2, x_3, x_4):** → They are the data/features you feed (e.g., height, weight, age).

•**Weights (w_1, w_2, w_3, w_4)**
•→ Strength of each connection (like synaptic strength in the brain).

Summation + Bias ($\Sigma w_i x_i + bias$)
•→ Like the cell body combining signals.
→ Bias shifts the decision boundary (like a neuron's internal threshold).

•**Activation Function f(x):** → Decides whether the neuron "fires" or not.
→It's a **step function** (1 or 0).
→ In modern ANN, we often use **ReLU, sigmoid, tanh** instead.

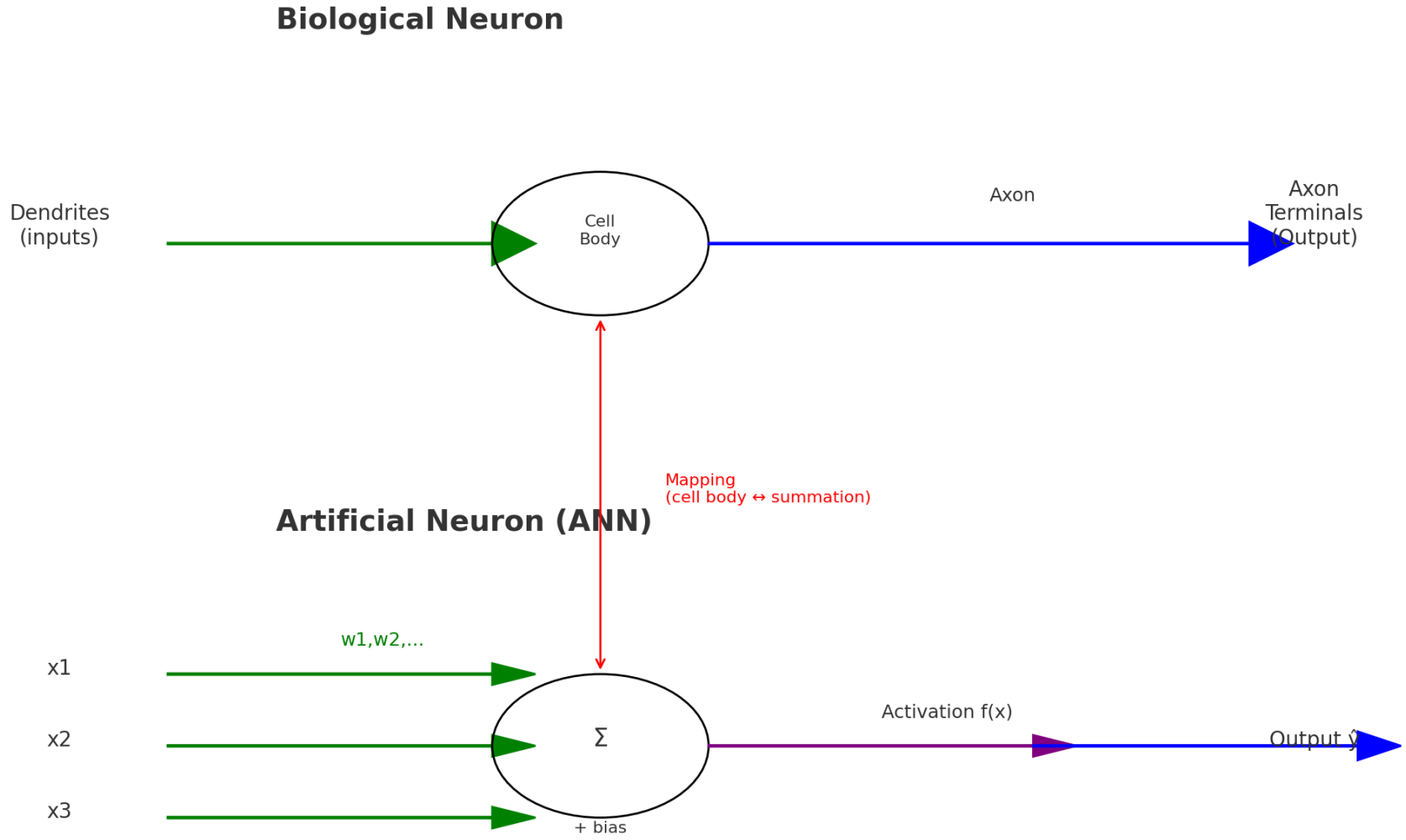
•**Output ($\hat{y}$)**
•→ Signal sent forward (like axon sending impulses).
→ Becomes input for the next neuron in the network.

Unit -1

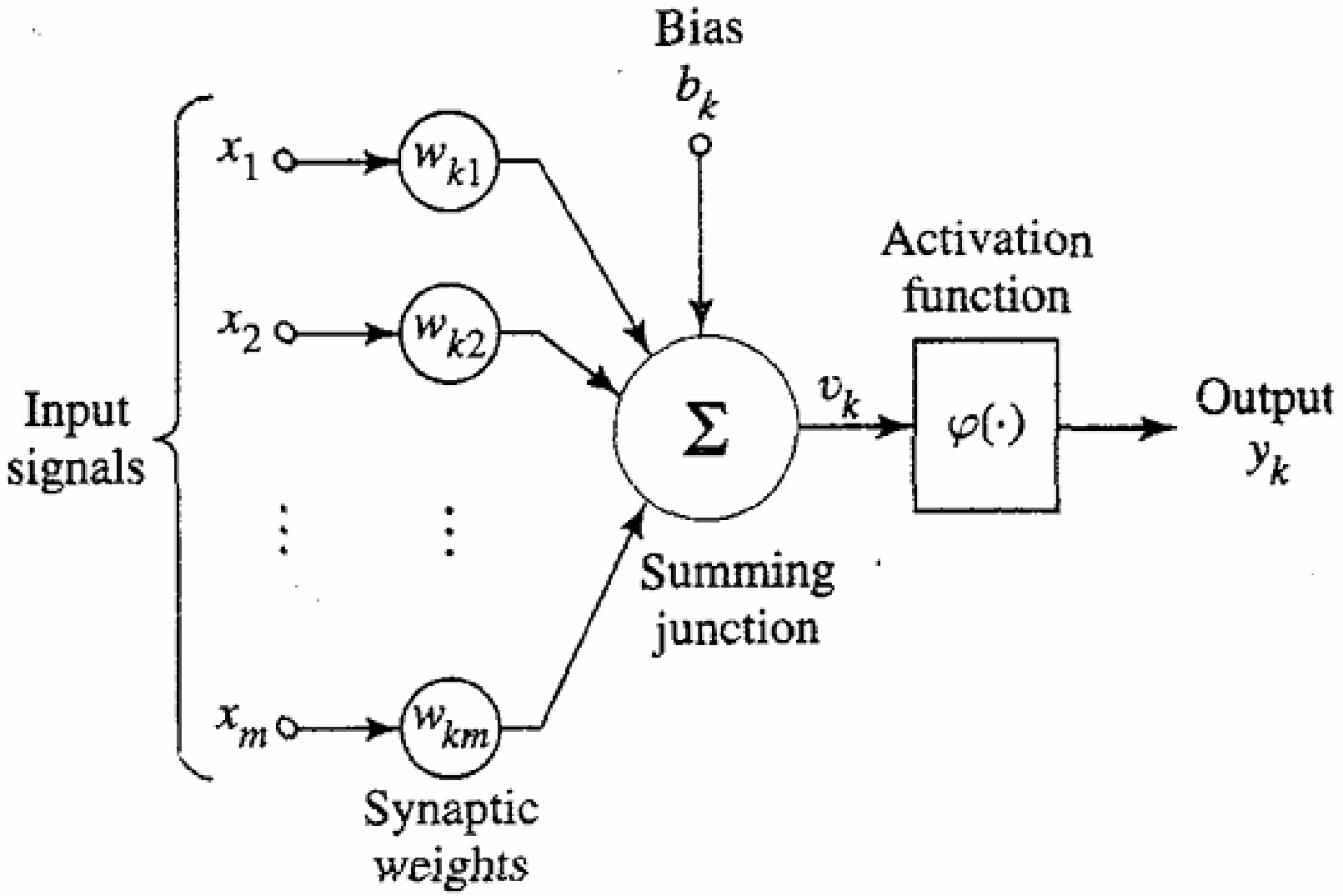
Mapping Between Biological and Artificial Neuron

Biological Neuron	Artificial Neuron (ANN)
Dendrites	Inputs ($x_1, x_2, \ldots, x_n$)
Synaptic Strengths	Weights ($w_1, w_2, \ldots, w_n$)
Cell Body (Soma)	Summation & Bias ($\Sigma w_i x_i + b$)
Firing Rule	Activation Function f(x)
Axon	Output ($\hat{y}$)
Axon Terminals	Input to next neurons

Mapping Between Biological and Artificial Neuron

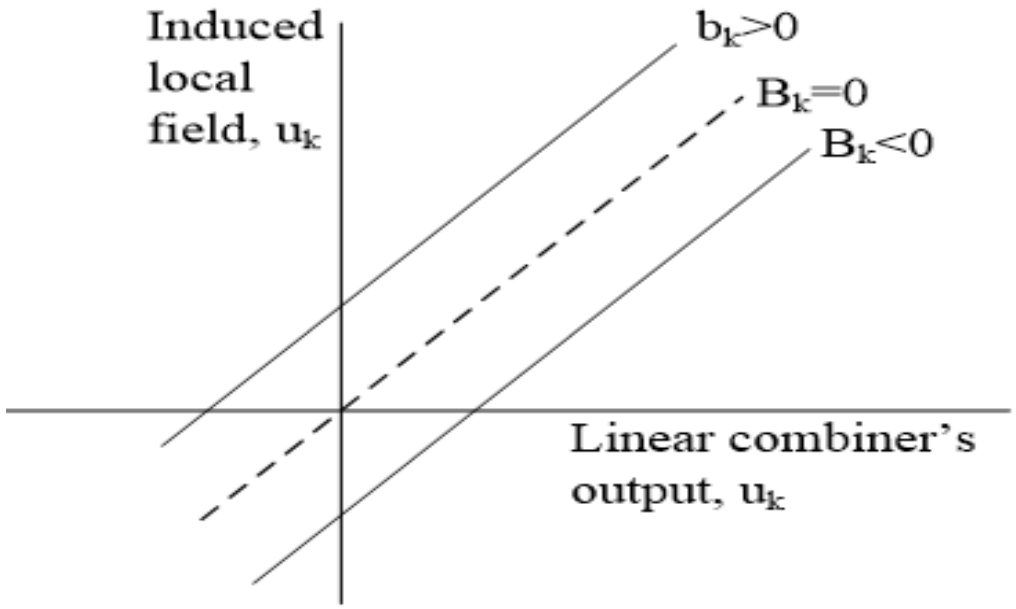


Unit -1

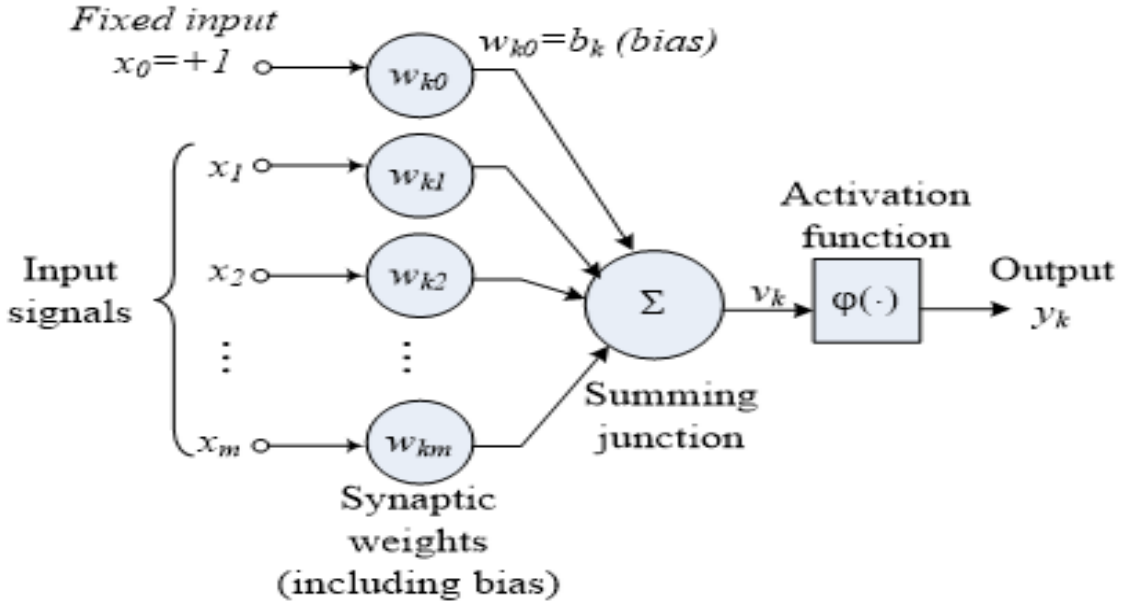

Bias
b_k
x_1
w_{k1}
x_2
w_{k2}
x_m
w_{km}
Input signals
Synaptic weights
Summing junction
v_k
Activation function
$\varphi(\cdot)$
Output
y_k

Unit -1

Non Linear Model of a Neuron



Affine transformation produced by the presence of a bias



Another Nonlinear model of a neuron

Unit -1

Functional Parameters of a Neuron

Function parameter	Description
w	Vector of real-valued weights on the connections
w · **x**	Dot product ($\sum_{i=1}^{n} w_i x_i$)
n	Number of inputs to the perceptron
b	The bias term (input value does not affect its value; shifts decision boundary away from origin)

Unit -1

Activation Function denoted by $\varphi(v)$ defines the output of a neuron in terms of induced local field v.

Threshold Function

$$\varphi(v) = \begin{cases} 1 & if\ v \geq 0 \\ 0 & if\ v < 0 \end{cases}$$

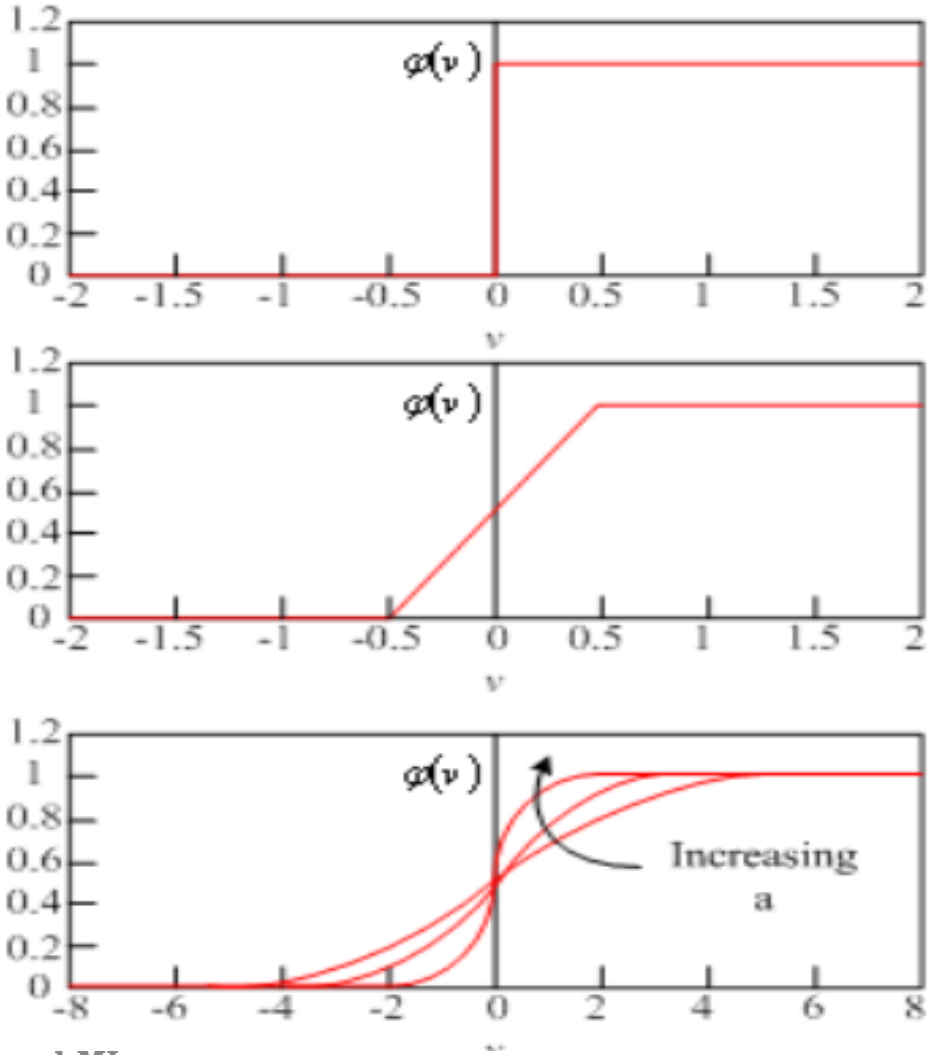
Piecewise-Linear Function

$$\varphi(v) = \begin{cases} 1 & v \geq 1/2 \\ v & -1/2 < v < 1/2 \\ 0 & v \leq -1/2 \end{cases}$$

Sigmoid Function

$$\varphi(v) = \frac{1}{1+\exp(-av)}$$

a is the slope parameter



Unit -1

- **Signum Function :** Activation function ranging from -1 to +1 where the activation function assumes an antisymmetric form with respect to origin, the threshold can be defined as

$$\varphi(v) = \begin{cases} 1 & \text{if } v > 0 \\ 0 & \text{if } v = 0 \\ -1 & \text{if } v < 0 \end{cases}$$

The hyperbolic tangent function of the same can be defined as $\varphi(v) = \tanh(v)$

Unit -1

- **Tanh (Hyperbolic Tangent):** $$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- **Range:** (–1, 1)
- **Pros:** Zero-centered (better than sigmoid for optimization).
- **Cons:** Still suffers from vanishing gradients for large |x|.

- **ReLU (Rectified Linear Unit)** $$f(x) = \max(0, x)$$
- **Range:** [0, ∞)
- **Pros:** Simple, computationally efficient, mitigates vanishing gradients, works well in deep nets.
- **Cons:** *Dying ReLU* problem (neurons stuck at 0 if weights push them negative).

Unit -1

- **Leaky ReLU:**

$$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

- (with small α, e.g., 0.01) - **Range:** (–∞, ∞)
- **Pros:** Fixes dying ReLU by allowing a small negative slope.
- **Cons:** α is a hyperparameter that needs tuning.

- **Softmax**
- Used for **multi-class classification** in the output layer.
- Converts logits into probabilities:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Which Activation to Use?

- **Hidden layers (general):** ReLU or its variants (Leaky ReLU, GELU).
- **Output layer:**
  - **Binary classification:** Sigmoid
  - **Multi-class classification:** Softmax
  - **Regression:** Linear (no activation)

Unit -1

Neural Networks as Directed Graphs – Signal Flow Diagrams

- **Neural Network (NN)** can be represented as a **directed graph.**
- Each node and edge in this graph represents a mathematical or functional relationship between signals.

- Block Diagram Representation
- The **block diagram** provides a *functional overview* of the neural network.
  - Each **block** represents a function or a neuron.
  - The **arrows** indicate the **direction of data flow**.
  - It provides a *high-level view* of the computation process.

Unit -1

Signal Flow Graph Representation

- The **signal flow graph (SFG)** gives a *complete mathematical and directional* representation of the neural network.
- It shows **nodes** and **directed edges (arrows)** between them.
- Each edge represents a *signal transmission* or *weight connection*.
- The arrows define the **direction of signal propagation** — typically from input to output.

- **(x1) → (Neuron 1) → (y1)**
- **(x2) → (Neuron 2) → (y2)**

Unit -1

Architectural Graph Representation

This diagram illustrates the layout or topology of the network, showing how the layers and neurons are interconnected.

Input Nodes → Hidden Nodes → Output Node

Explanation:

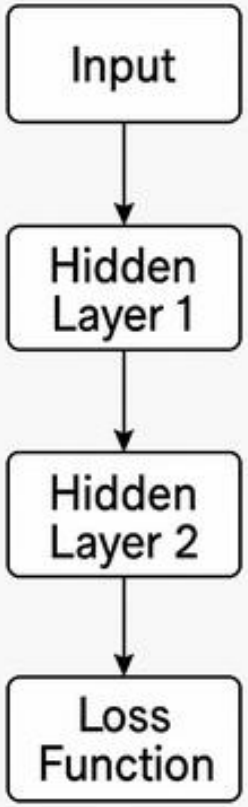
Input layer: Nodes that take input signals (x_1, x_2, …).

Hidden layer(s): Nodes that process signals via activation functions.

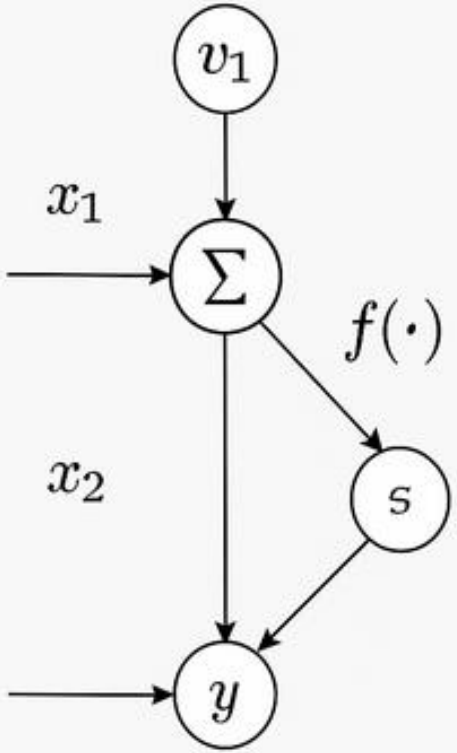
Output layer: Produces the final result or decision.

Unit -1

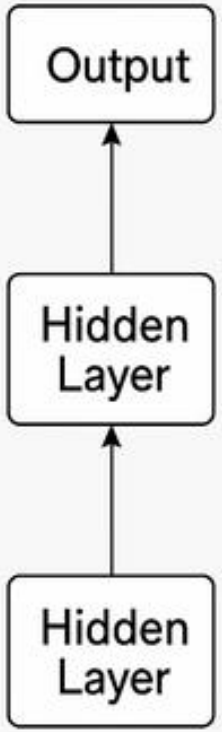
Block Diagram



Signal Flow Graph



Architectural Graph



Unit -1

Description of the Graph

Source Nodes:

These supply **input signals** to the network (e.g., features like x_1, x_2). They represent where data enters the system.

Computation Nodes:

Each **neuron** is represented as a single node that performs computation.

These are sometimes called **processing elements**.

Each node performs:

$$y_j = f\left(\sum_i w_{ji} x_i + b_j\right)$$

Unit -1

Communication Links:

The **directed edges (arrows)** connecting nodes define the **path of information flow**.

They:
Indicate **direction** of data (from input → output).

Carry **no weight themselves** (weights belong to nodes).

Represent **dependencies** between neurons.

$$h_1 = f(w_{11}x_1 + w_{12}x_2 + b_1)$$

$$h_2 = f(w_{21}x_1 + w_{22}x_2 + b_2)$$

$$y = f(v_1h_1 + v_2h_2 + b_3)$$

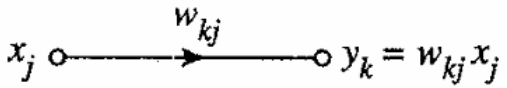
Unit -1

Neural Networks as Directed Graphs – Signal Flow Diagrams

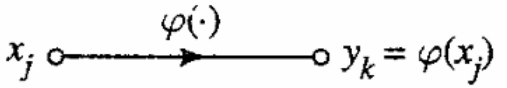
Rule 1. A signal flows along a link only in the direction defined by the arrow on the link.

Two different types of links may be distinguished:

- *Synaptic links*, whose behavior is governed by a *linear* input–output relation. Specifically, the node signal x_j is multiplied by the synaptic weight w_{kj} to produce the node signal y_k, as illustrated in Fig. 1.9a.
- *Activation links*, whose behavior is governed in general by a *nonlinear* input–output relation. This form of relationship is illustrated in Fig 1.9b, where $\varphi(\cdot)$ is the nonlinear activation function.



(a)



(b)

Unit -1

Neural Networks as Directed Graphs – Signal Flow Diagrams

Simple Example

Suppose:

- Inputs: $x_1 = 1$,$x_2 = 2$
- Weights: $w_1 = 0.5$,$w_2 = 0.2$
- Activation function:

Step 1 – Synaptic (Linear):

$$z = (0.5)(1) + (0.2)(2) = 0.9$$

Step 2 – Activation (Nonlinear):

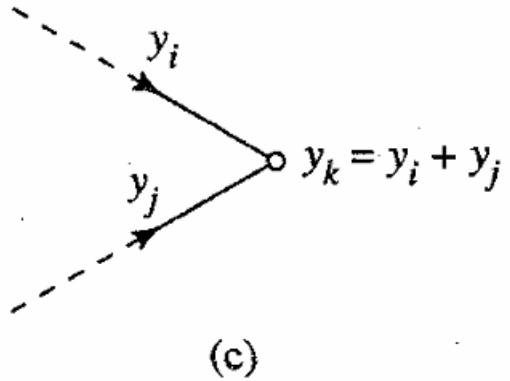
$$y = ReLU(0.9) = 0.9$$

Unit -1

Neural Networks as Directed Graphs

Rule 2. A node signal equals the algebraic sum of all signals entering the pertinent node via the incoming links.

This second rule is illustrated in Fig. 1.9c for the case of *synaptic convergence* or *fan-in*.



(c)

Unit -1

Neural Networks as Directed Graphs

Example

Let's take an example neuron with three inputs:

Input	Weight	Weighted Input
(x_1 = 1)	(w_1 = 0.4)	(0.4)
(x_2 = 2)	(w_2 = 0.6)	(1.2)
(x_3 = -1)	(w_3 = 0.5)	(-0.5)

Now sum all weighted inputs:

$$v = 0.4 + 1.2 - 0.5 = 1.1$$

Then, apply an activation function (say ReLU): y=max(0,1.1)=1.1

Unit -1

Neural Networks as Directed Graphs

Rule 3. The signal at a node is transmitted to each outgoing link originating from that node, with the transmission being entirely independent of the transfer functions of the outgoing links.

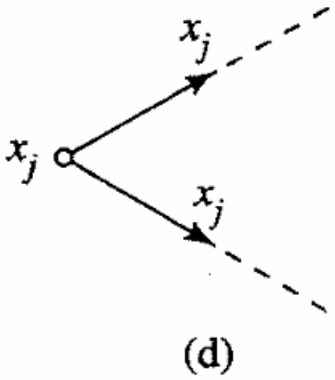
This graph illustrates the synaptic divergence or fanout

Mathematically

If neuron **A** has an output y_A ,and it connects to neurons **B** and **C** with weights $w_{BA}$and w_{CA} ,then:

$$\text{Input } to\ B\text{: } x_B = w_{BA} \cdot y_A$$

$$\text{Input } to\ C\text{: } x_C = w_{CA} \cdot y_A$$



Here:

- $y_A$is the same for both links.
- The weights $w_{BA}$and $w_{CA}$are **independent**.
- The **transfer (activation) functions** of neurons **B** and **C** are separate — neuron **A** doesn't influence them.

Unit -1

Neural Networks as Directed Graphs

Example

Suppose a neuron produces output $y = 0.8$ and sends it to three neurons via different weights:

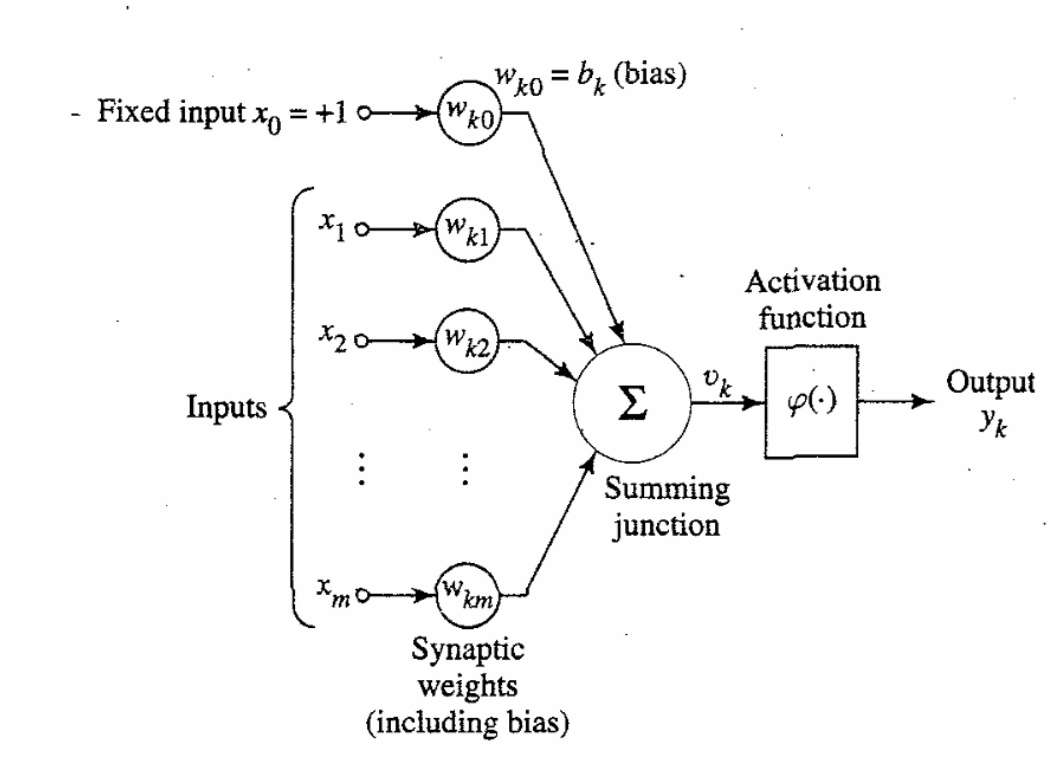
Outgoing Link	Weight	Signal Sent
To Neuron 1	0.5	(0.8 * 0.5 = 0.4)
To Neuron 2	0.2	(0.8 * 0.2 = 0.16)
To Neuron 3	1.0	(0.8 * 1.0 = 0.8)

The **original output (0.8)** was transmitted to all links,

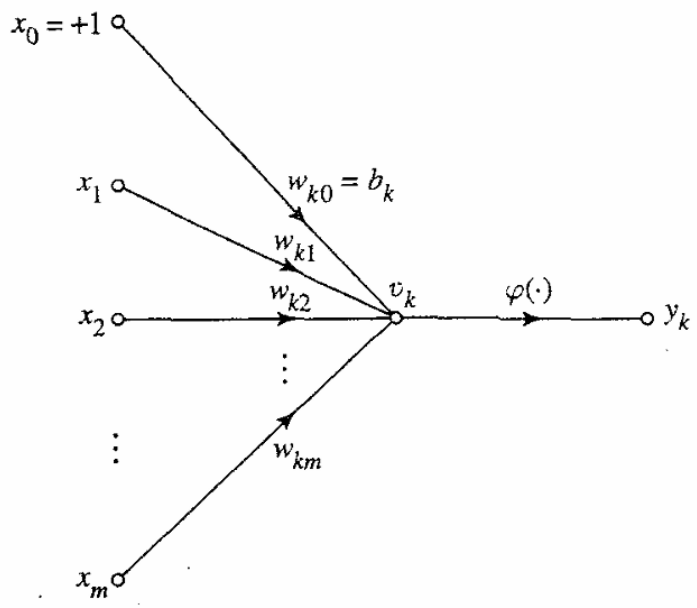
but each **receiving neuron** got a **different weighted version** of it.

Unit -1

Neural Networks as Directed Graphs



Neural Network Diagram

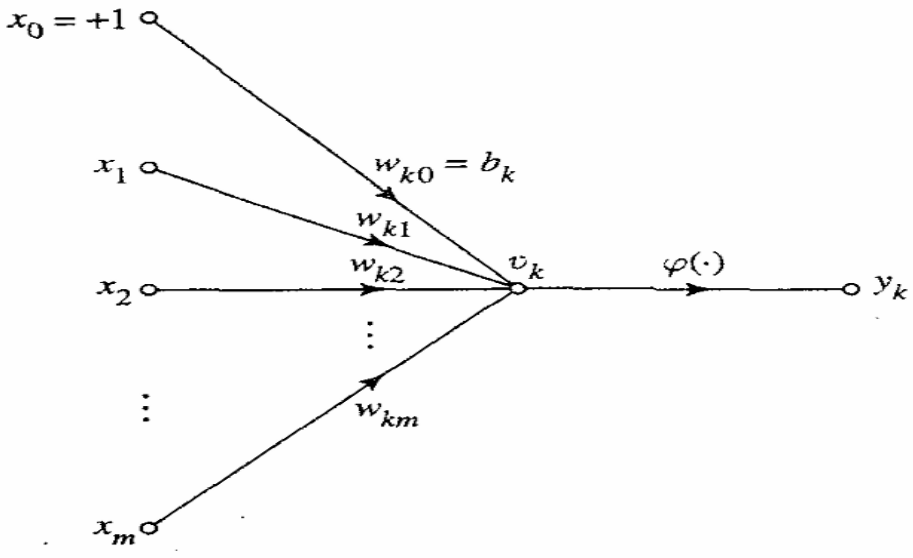


Signal Flow Diagram

Unit -1

Neural Networks as Directed Graphs

Mathematical definition of a NN



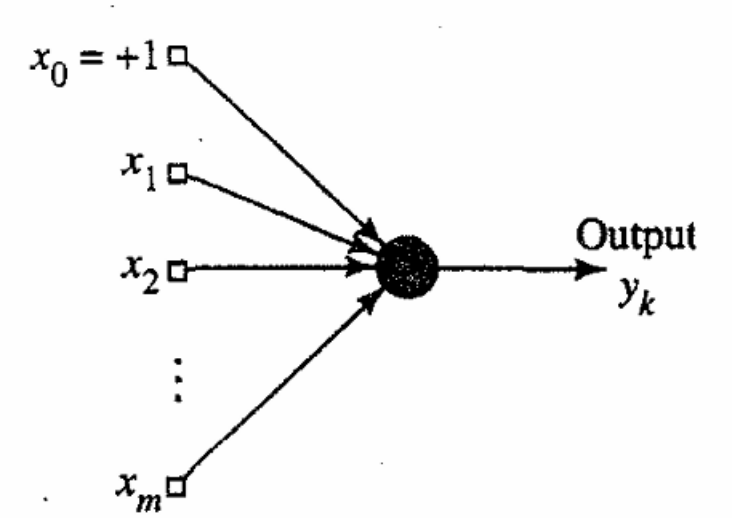
A neural network is a directed graph consisting of nodes with interconnecting synaptic and activation links, and is characterized by four properties:

1. *Each neuron is represented by a set of linear synaptic links, an externally applied bias, and a possibly nonlinear activation link. The bias is represented by a synaptic link connected to an input fixed at +1.*
2. *The synaptic links of a neuron weight their respective input signals.*
3. *The weighted sum of the input signals defines the induced local field of the neuron in question.*
4. *The activation link squashes the induced local field of the neuron to produce an output.*

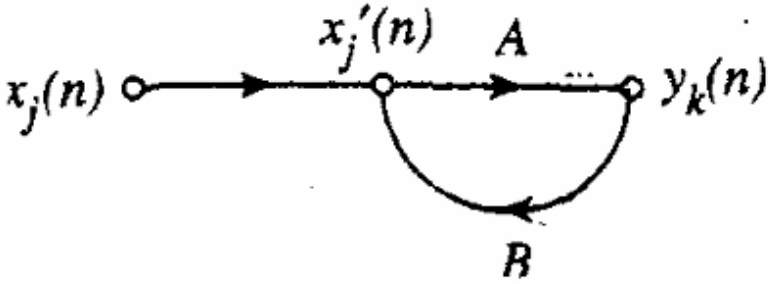
Unit -1

Feedback

Feedbacks are special class of neural networks called as Recurrent Neural Networks (RNNs)



Architectural Graph



Signal Flow Graph

Unit -1

Feedback

Example – Simple Recurrent Computation

Suppose:

$$y(n) = 0.5 \cdot x(n) + 0.4 \cdot y(n-1)$$

If:

$$x(1) = 2$$
$$y(0) = 0$$

Then:

$$y(1) = 0.5(2) + 0.4(0) = 1$$

If next input $x(2) = 1$:

$$y(2) = 0.5(1) + 0.4(1) = 0.9$$

→The output at time *n* depends on both *current* and *previous* values.

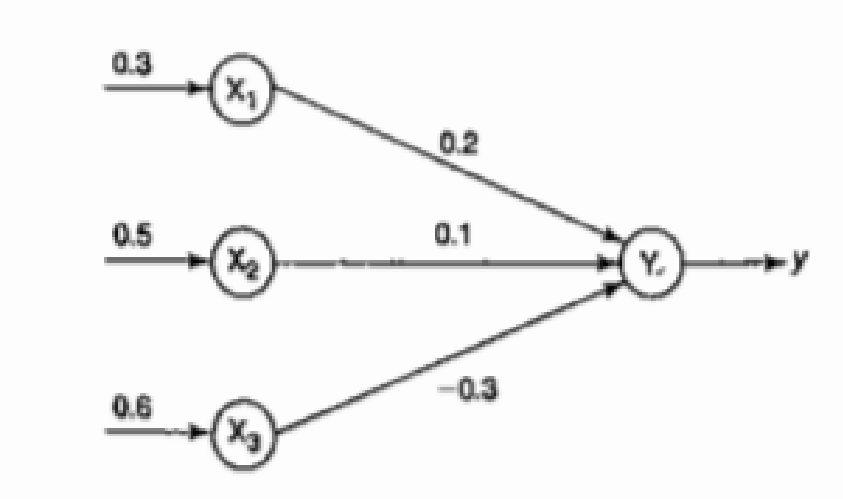
Unit -1

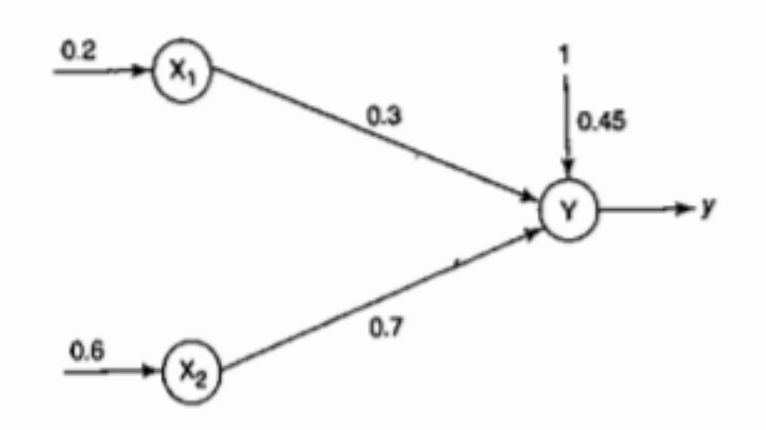
Concept	Feedforward Network	Feedback (Recurrent) Network
Flow Direction	Only forward	Forward + backward loop
Memory	No memory	Has memory of previous outputs
Graph Type	Directed acyclic	Directed cyclic (loop exists)
Equation	(y = f(Wx))	(y(n) = f(Ax(n) + Ry(n-1)))
Applications	Classification, regression	Speech, text, time-series, control

Unit -1

Problems

- For the given network, calculate the net input to the output neuron



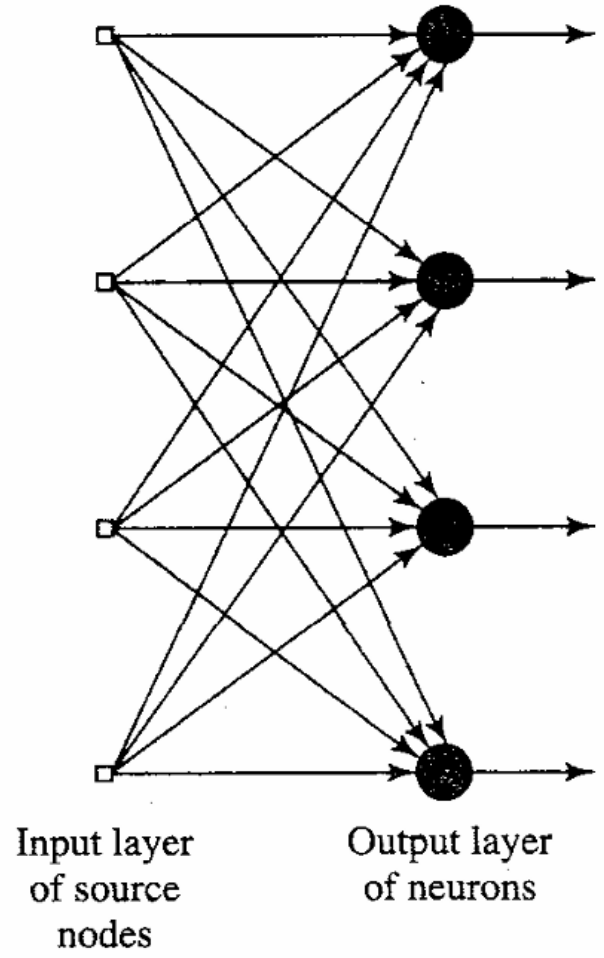


- Calculate the net input (i) Without bias and (ii) when the bias is 0.45 and observe the difference and discuss

Unit -1

Network Architeures

- Single Layer Feedforward network
  - Consists of input layer of source nodes that projects to output layer of computation node
  - Strictly feedforward or acylic type
  - Single output layer of computation nodes
- Layer is formed by taking a processing element and combining it with other processing elements
- Interconnections lead to the formation of various network architectures



Unit -1

Problem 1: Single-Layer Feedforward Neural Network: **Given:** A single neuron receives three inputs:

Input	(x_i)	Weight (w_i)
(x_1)	0.5	0.4
(x_2)	0.2	0.3
(x_3)	0.8	0.6

Bias $b = 0.1$

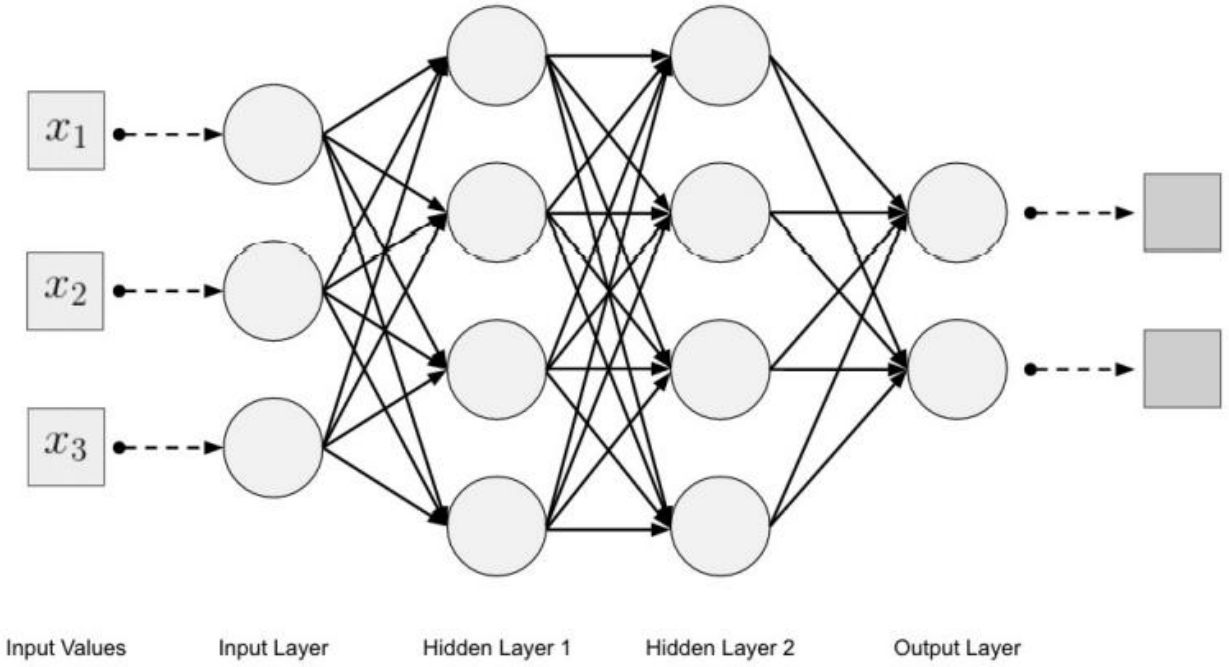
Activation Function:

$$f(v) = \frac{1}{1 + e^{-v}} \text{ (Sigmoid function)}$$

Unit -1

Network Architeures

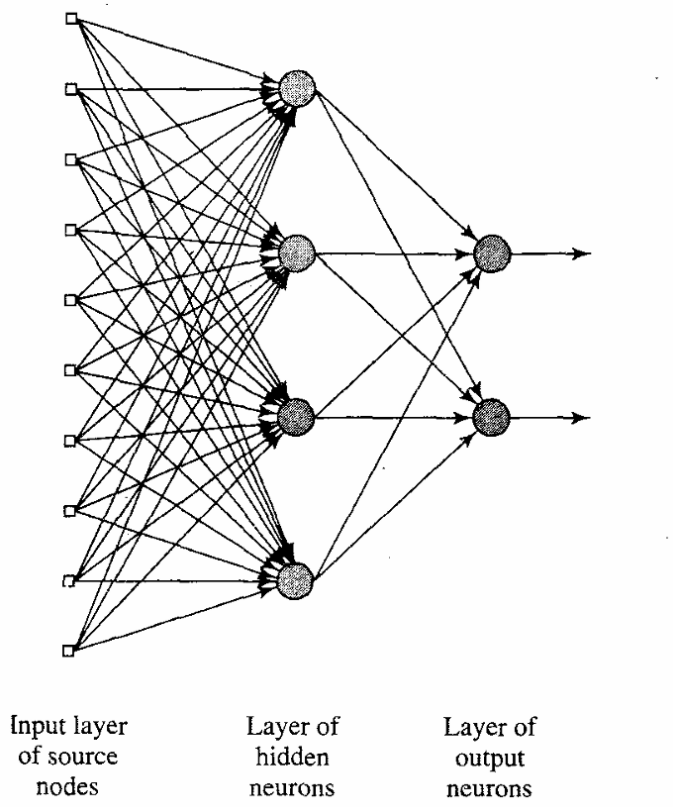
- Multi Layer Feedforward network



Unit -1

Network Architeures

- Multi Layer Feedforward network
  - Consists of input layer , hidden layer and output layer
- Each layer can have a different number of neurons and each layer is fully connected to the adjacent layer.
- The connections between the neurons in the layers form an acyclic graph



Unit -1

Problem 2: Multi-Layer Feedforward Neural Network: Given: A two-layer network (1 hidden neuron and 1 output neuron)

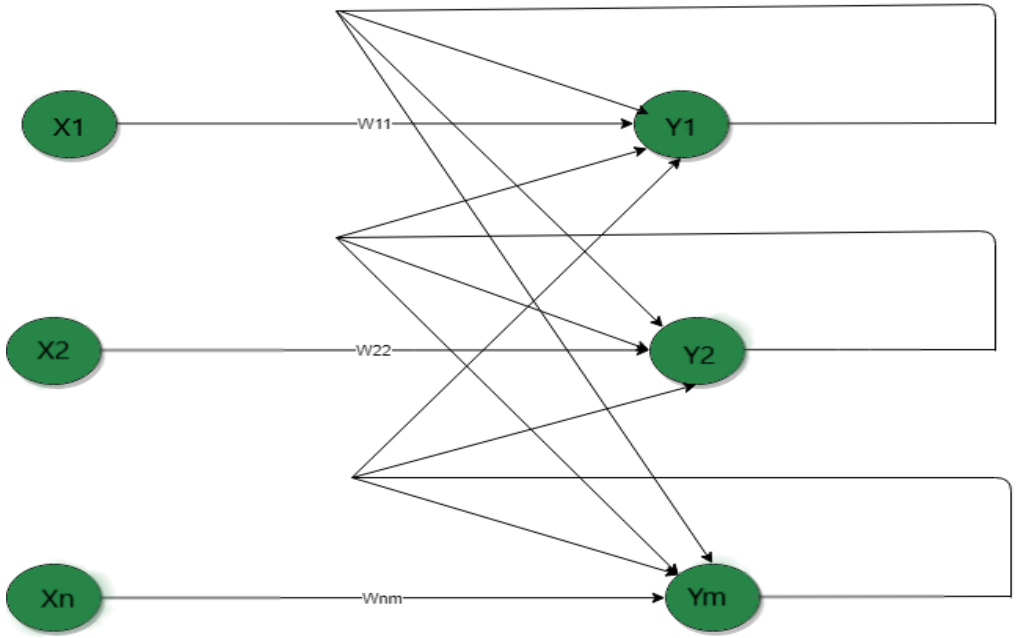
Layer	Inputs	Weights	Bias
Hidden Neuron (h_1)	(x_1=0.4, x_2=0.6)	(w_{11}=0.5, w_{12}=0.7)	(b_1=0.2)
Output Neuron (y)	(h_1)	(w_{21}=0.9)	(b_2=0.1)

Activation: Sigmoid for both layers.

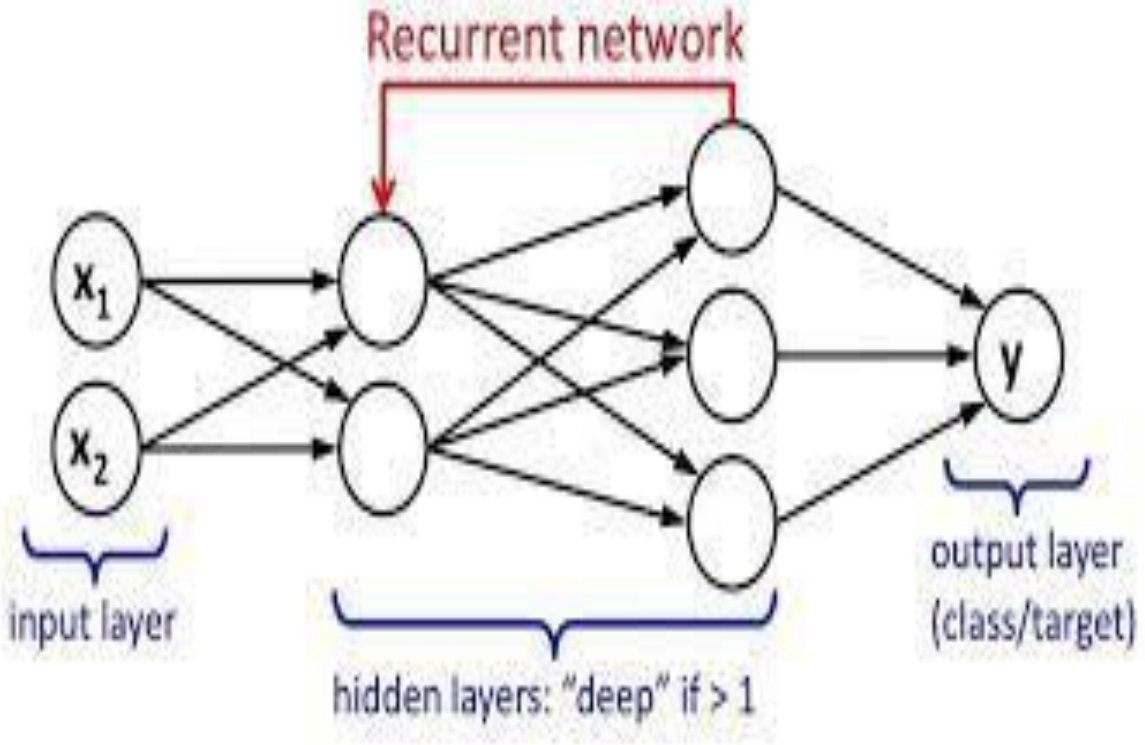
Unit -1

Recurrent Neural Network(RNN) - the output from the previous step is fed as input to the current step.

Single Layer Recurrent Network

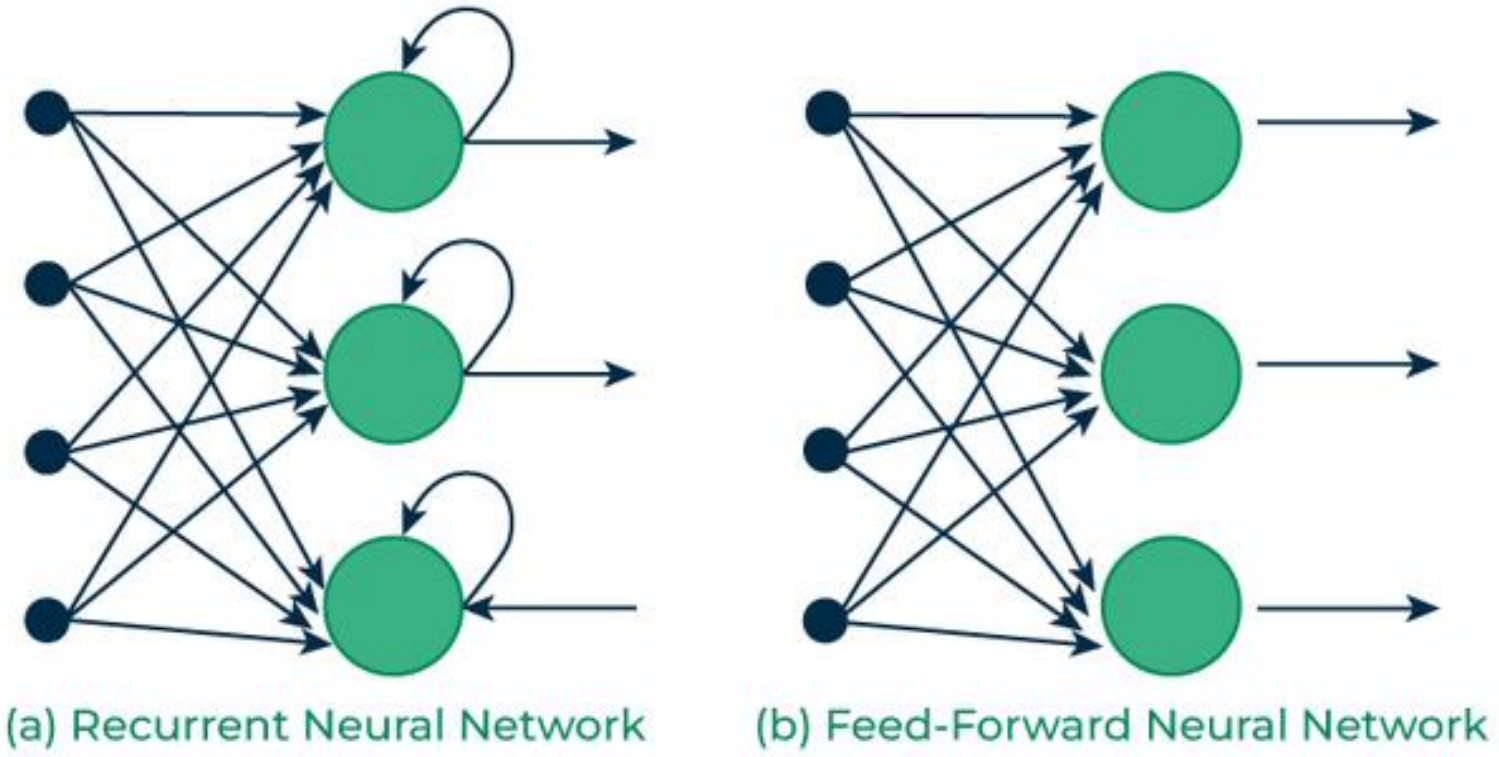


Multi Layer Recurrent Network



Unit -1

Recurrent Neural Networks Vs Feed forward Networks



(a) Recurrent Neural Network

(b) Feed-Forward Neural Network

Unit -1

Numerical Example: Backpropagation in a 2–1 Network: Network Structure

We'll use a small **feedforward neural network**:

- Inputs: $x_1 = 0.05,\ x_2 = 0.10$
- Weights: $w_1 = 0.15,\ w_2 = 0.20$
- Bias: $b = 0.35$
- Target output (desired): $t = 0.01$
- Learning rate (η): 0.5
- Activation function: Sigmoid $f(v) = \frac{1}{1+e^{-v}}$

Unit -1

Learning Process

Error Correction Learning

Memory Based Learning

Hebbian Learning

Competitive Learning

Boltzmann Learning

Learning with Teacher

Learning without Teacher

Unit -1

Error Correction Learning

Consider a simple case of a neuron k constituting the only computational node in the output layer of a feedforward neural network

Neuron k is driven by a signal vector x(n) produced by one or more hidden layers

Hidden layers are driven by an input vector applied from source node

The argument n denotes the discrete time , the time step of an iterative process involved in adjusting the synaptic weights of neuron k

The output signal of the neuron k is denoted by $y_k(n)$

The output signal is comparted to desired response or target output *dk(n).*

An error correction an error signal, denoted by *ek(n),* is produced

$$e_k(n) = d_k(n) - y_k(n)$$

Error Correction Learning - Example

The error signal ek(n) actuates a control mechanism, apply a sequence of corrective adjustments to the synaptic weights of neuron k.

The corrective adjustments are designed to make the output signal *Yk(n)* come closer to the desired

response *dk(n)* in a step-by-step manner. This objective is achieved by minimizing a *cost function* or *index of performance dk(n),*defined in terms of the error signal *ek(n)*

Unit -1

Types of Learning

Memory-based Learning (instance-based)

Concept: The system stores training examples and makes predictions based on similarity to previously stored instances.

No explicit model is formed — learning is "lazy."

Algorithm Example:
k-Nearest Neighbors (k-NN):
Store all data points (x_i, y_i)
For a new input x' ,compute distance to all stored samples
Predict based on the majority label (classification) or average output (regression)

Unit -1

Types of Learning

Hebbian Learning – Overview

- Hebbian Learning is one of the **oldest and most important learning rules in neural networks**.

•

It was proposed by psychologist **Donald Hebb (1949)** and is often summarized by the phrase:

- **"Cells that fire together, wire together."**

- This means that **if two neurons are active at the same time**, the connection (synapse) between them becomes **stronger**.

•

In other words, learning happens when the activity of one neuron repeatedly helps trigger another neuron.

Unit -1

Types of Learning

If neuron **A** helps activate neuron **B**, the connection weight between them (**w**) is increased:

$$\Delta w = \eta \, x \, y$$

where

Δw= change in connection weight

η= learning rate

x= input signal (activity of neuron A)

y= output signal (activity of neuron B)

Unit -1

Types of Learning

Hebbian Learning

Properties of Hebbian Synapse

1. Time Dependent Mechanism – exact timing of activations matter.
2. Local Mechanism – local signals – x and y (input and output) – no global information is needed.
3. Interactive Mechanism - Interaction between neurons

- Both neurons must be active; if either is inactive, no learning occurs. Hence, interaction is essential for weight modification.

4. Conjunctional or correlational Mechanism - **correlation** between the input and output signals.

- If the two are positively correlated (fire together), the weight increases. If they are negatively correlated (fire at different times), the weight decreases.

Unit -1

Types of Learning

Mathematical Model of Hebbian Network

To formulate Hebbian learning in mathematical terms, consider a synaptic weight w_{kj} of neuron k with presynaptic and postsynaptic signals denoted by x_j and y_k, respectively. The adjustment applied to the synaptic weight w_{kj} at time step n is expressed in the general form

$$\Delta w_{kj}(n) = F(y_k(n), x_j(n))$$

where $F(\cdot,\cdot)$ is a function of both postsynaptic and presynaptic signals. The signals $x_j(n)$ and $y_k(n)$ are often treated as dimensionless.

Form 1:

Hebb's hypothesis. The simplest form of Hebbian learning is described by

$$\Delta w_{kj}(n) = \eta y_k(n) x_j(n) \qquad (2.9)$$

where η is a positive constant that determines the *rate of learning*.

Unit -1

Types of Learning

Dis advantage of Hebb's Hypothesis

- The repeated application of the input signal (presynaptic activity) xj leads to an increase in yk and therefore exponential grown that drives the synaptic connection into saturation
- No information will be stored in synapse and the selectivity is lost

Form 2 :

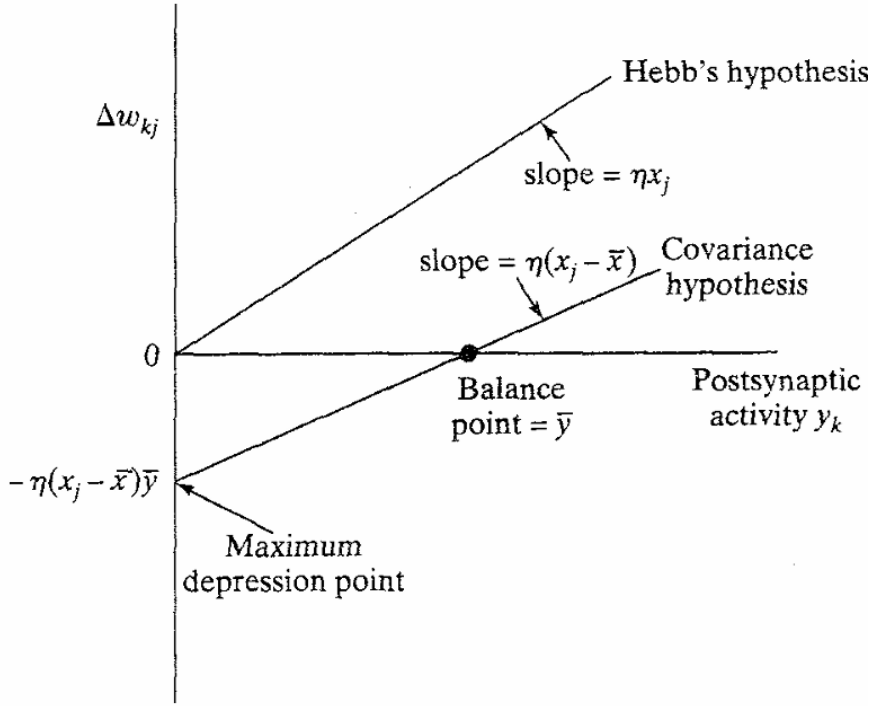
Covariance hypothesis. One way of overcoming the limitation of Hebb's hypothesis is to use the *covariance hypothesis* introduced in Sejnowski (1977a, b). In this hypothesis, the presynaptic and postsynaptic signals in Eq. (2.9) are replaced by the departure of presynaptic and postsynaptic signals from their respective average values over a certain time interval. Let $\bar{x}$ and $\bar{y}$ denote the *time-averaged values* of the presynaptic signal x_j and postsynaptic signal y_k, respectively. According to the covariance hypothesis, the adjustment applied to the synaptic weight w_{kj} is defined by

$$\Delta w_{kj} = \eta(x_j - \bar{x})(y_k - \bar{y}) \qquad (2.10)$$

Unit - 1

Types of Learning

Form 2 :



covariance hypothesis allows for the following:

- Convergence to a nontrivial state, which is reached when $x_k = \bar{x}$ or $y_j = \bar{y}$.
- Prediction of both synaptic *potentiation* (i.e., increase in synaptic strength) and synaptic *depression* (i.e., decrease in synaptic strength).

Unit -1

Types of Learning

Form 2 :

covariance hypothesis allows for the following:

- Convergence to a nontrivial state, which is reached when $x_k = \bar{x}$ or $y_j = \bar{y}$.
- Prediction of both synaptic *potentiation* (i.e., increase in synaptic strength) and synaptic *depression* (i.e., decrease in synaptic strength).

Unit -1

Types of Learning

Observations from eqn 2.10

1. Synaptic weight w_{kj} is enhanced if there are sufficient levels of presynaptic and postsynaptic activities, that is, the conditions $x_j > \bar{x}$ and $y_k > \bar{y}$ are both satisfied.
2. Synaptic weight w_{kj} is depressed if there is either
   - a presynaptic activation (i.e., $x_j > \bar{x}$) in the absence of sufficient postsynaptic activation (i.e., $y_k < \bar{y}$), or
   - a postsynaptic activation (i.e., $y_k > \bar{y}$) in the absence of sufficient presynaptic activation (i.e., $x_j < \bar{x}$).

Unit -1

Types of Learning

Competitive Learning

Basic Elements of Competitive Learning

- A set of neurons that are all the same except for some randomly distributed synaptic weights, and which therefore *respond differently* to a given set of input patterns.
- A *limit* imposed on the "strength" of each neuron.
- A mechanism that permits the neurons to *compete* for the right to respond to a given subset of inputs, such that only *one* output neuron, or only one neuron per group, is active (i.e., "on") at a time. The neuron that wins the competition is called a *winner-takes-all neuron.*

Accordingly the individual neurons of the network learn to specialize on ensembles of similar patterns; in so doing they become *feature detectors* for different classes of input patterns.

Unit -1

Types of Learning

Competitive Learning

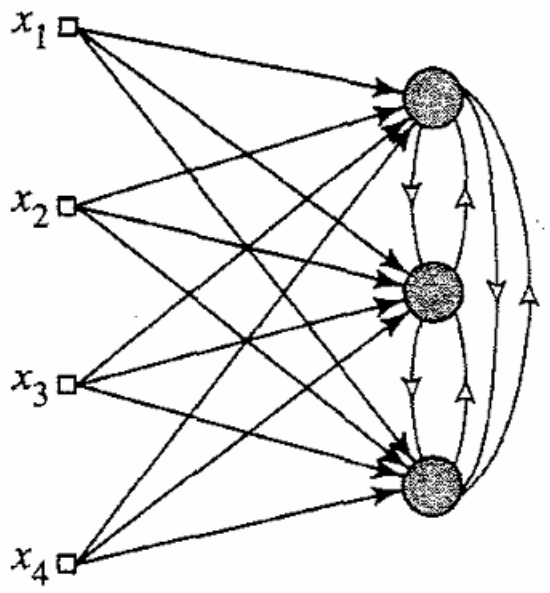


FIGURE 2.4 Architectural graph of a simple competitive learning network with feedforward (excitatory) connections from the source nodes to the neurons, and lateral (inhibitory) connections among the neurons; the lateral connections are signified by open arrows.

Unit -1

Types of Learning

Competitive Learning

- Single Layer of output neurons each is connected to input nodes
- May include feedback connections among neurons
- Preform lateral inhibition with each neuron tending to inhibit the neuron that is laterally connected

$$y_k = \begin{cases} 1 & \text{if } v_k > v_j \text{ for all } j, j \neq k \\ 0 & \text{otherwise} \end{cases} \qquad \sum_j w_{kj} = 1 \quad \text{for all } k$$

According to the standard *competitive learning rule*, the change Δw_{kj} applied to synaptic weight w_{kj} is defined by

$$\Delta w_{kj} = \begin{cases} \eta(x_j - w_{kj}) & \text{if neuron } k \text{ wins the competition} \\ 0 & \text{if neuron } k \text{ loses the competition} \end{cases} \qquad (2.13)$$

where η is the learning-rate parameter. This rule has the overall effect of moving the synaptic weight vector $\mathbf{w}_k$ of winning neuron k toward the input pattern $\mathbf{x}$.

Unit -1

Types of Learning

Boltzmann Learning

- Neural network built in the name of Boltzman is called as Boltzmann Learning
- The neurons constitute a recurrent structure and operate in binary manner
- They are either in “on” state or “off” state.
- The energy function is given by

$$E = -\frac{1}{2}\sum_{j}\sum_{\substack{k \\ j \neq k}} w_{kj} x_k x_j$$

where xj is the state of the neuron j and wkj is the synaptic weight connecting neuron j to k.

Unit -1

Types of Learning

Boltzmann Learning

. The machine operates by choosing a neuron at random—for example, neuron k—at some step of the learning process, then flipping the state of neuron k from state x_k to state $-x_k$ at some temperature T with probability

$$P(x_k \rightarrow -x_k) = \frac{1}{1 + \exp(-\Delta E_k / T)} \qquad (2.16)$$

where ΔE_k is the *energy change* (i.e., the change in the energy function of the machine)

Unit -1

Types of Learning

Boltzmann Learning – two functional groups: visible and hidden

- Visible neurons provide an interface between the network and the environment that it operates in
- Hidden neurons always operate freely

- **Two modes of operation**
- Clamped Condition – in which visible neurons are all clamped onto specific states determined by the environment
- Free running condition: in which all neurons are allowed to operate freely

Unit -1

Types of Learning

Boltzmann Learning – two functional groups : visible and hidden

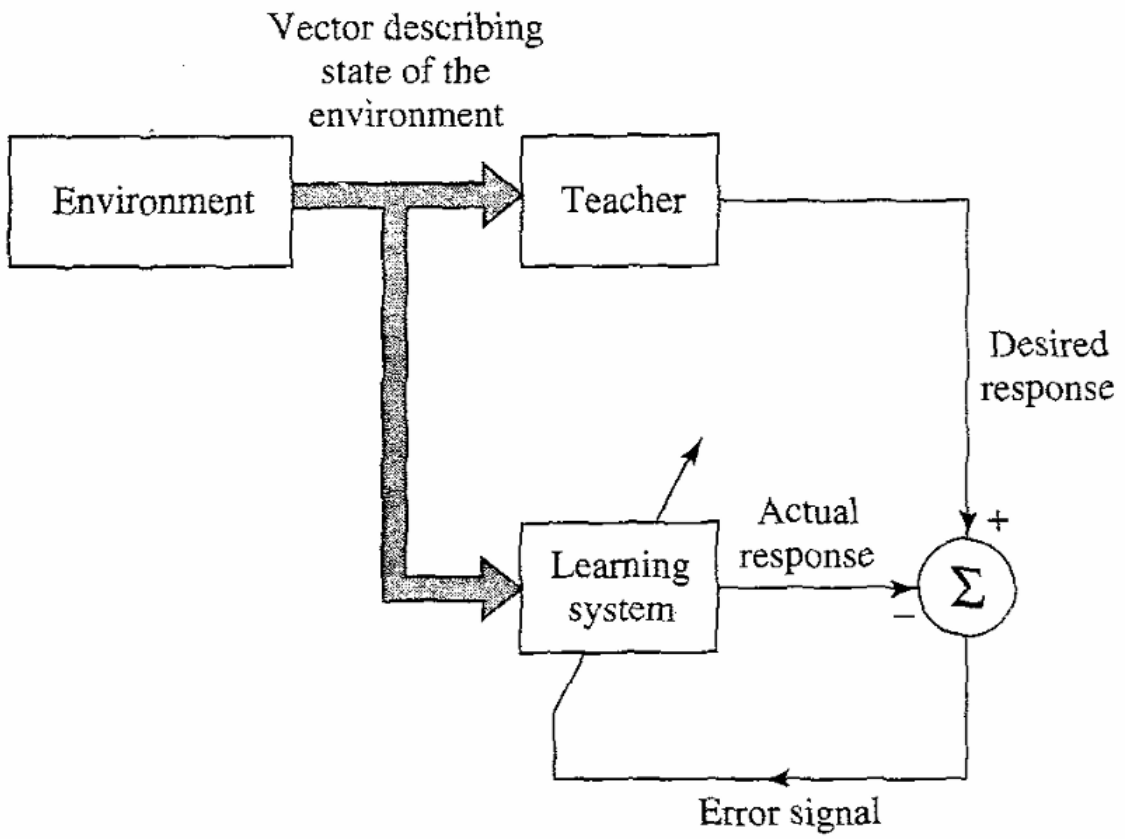
Let ρ_{kj}^{+} denote the *correlation* between the states of neurons j and k, with the network in its clamped condition. Let ρ_{kj}^{-} denote the *correlation* between the states of neurons j and k with the network in its free-running condition. Both correlations are averaged over all possible states of the machine when it is in thermal equilibrium. Then, according to the *Boltzmann learning rule*, the change Δw_{kj} applied to the synaptic weight w_{kj} from neuron j to neuron k is defined by (Hinton and Sejnowski, 1986)

$$\Delta w_{kj} = \eta(\rho_{kj}^{+} - \rho_{kj}^{-}), \qquad j \neq k \tag{2.17}$$

Unit -1

Types of Learning

Learning with a Teacher

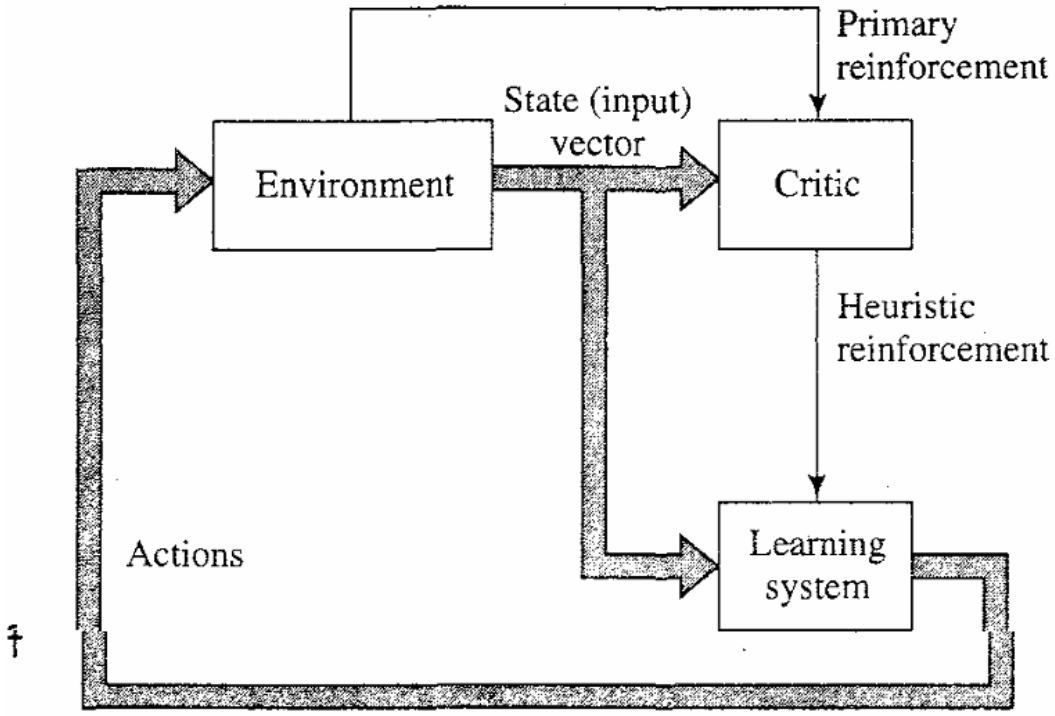


Unit -1

Types of Learning

Learning without a Teacher

1. Reinforcement Learning



Unit -1

Types of Learning

Aspect	Supervised Learning	Reinforcement Learning
Teacher	Provides correct output	No teacher, only feedback
Input	Labeled data	State from environment
Output	Correct label	Best action to maximize reward
Feedback	Error signal	Reward or penalty
Goal	Minimize error	Maximize cumulative reward

Unit -1

Types of Learning

Reinforcement Learning

- The system is designed to learn under delayed reinforcement , ie the system observes a temporal sequence of stimul recd from environment
- Goal of learning is to minimize a cost-to-go function , defined as the expectation of cumulative cost of actions taken over a sequence of steps instead of immediate cost

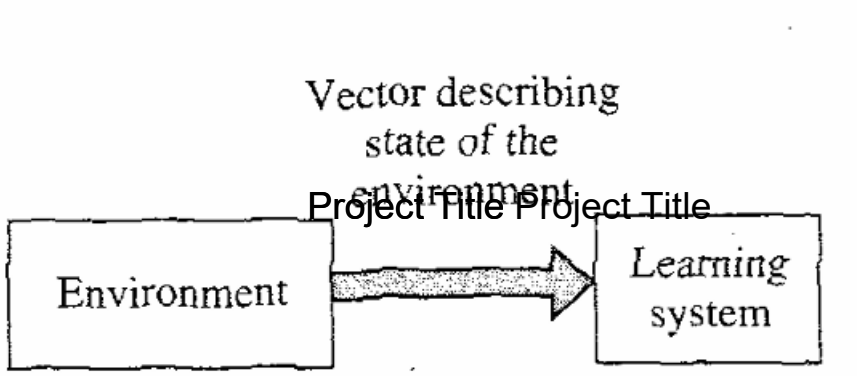
Delayed-reinforcement learning is difficult to perform for two basic reasons:

- There is no teacher to provide a desired response at each step of the learning process.
- The delay incurred in the generation of the primary reinforcement signal implies that the learning machine must solve a *temporal credit assignment problem*. By this we mean that the learning machine must be able to assign credit and blame individually to each action in the sequence of time steps that led to the final out-come, while the primary reinforcement may only evaluate the outcome.

Unit -1

Types of Learning

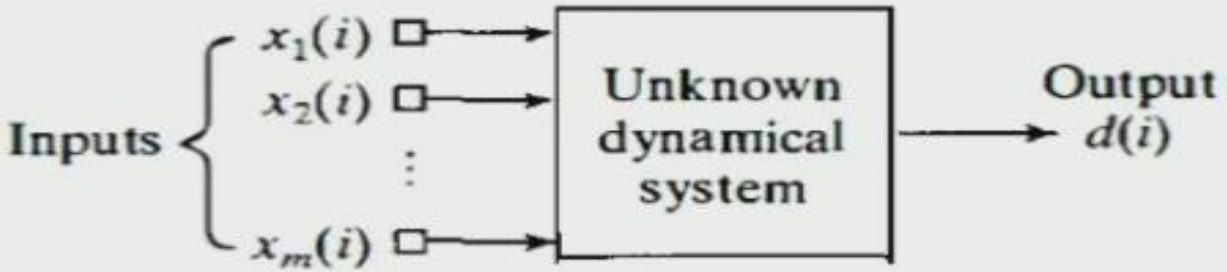
Unsupervised Learning



Adaptive Filter Problem

Consider a dynamic system with an unknown mathematical characterization

A set of labeled input-output data generated by the system at discrete instants of time at some uniform rate is available



Thus, the external behaviour of the system is described by the data set

$$\mathcal{T}: \{\mathbf{x}(i), d(i); i = 1, 2, \ldots, n, \ldots\}$$

Where,

$$\mathbf{x}(i) = [x_1(i), x_2(i), \ldots, x_m(i)]^T$$

The stimulus x(i) can arise in one of two fundamentally different ways. one spatial and the other temporal:

- The m elements of $\mathbf{x}(i)$ originate at different points in space; in this case we speak of $\mathbf{x}(i)$ as a *snapshot* of data.
- The m elements of $\mathbf{x}(i)$ represent the set of present and $(m - 1)$ past values of some excitation that are *uniformly spaced in time.*

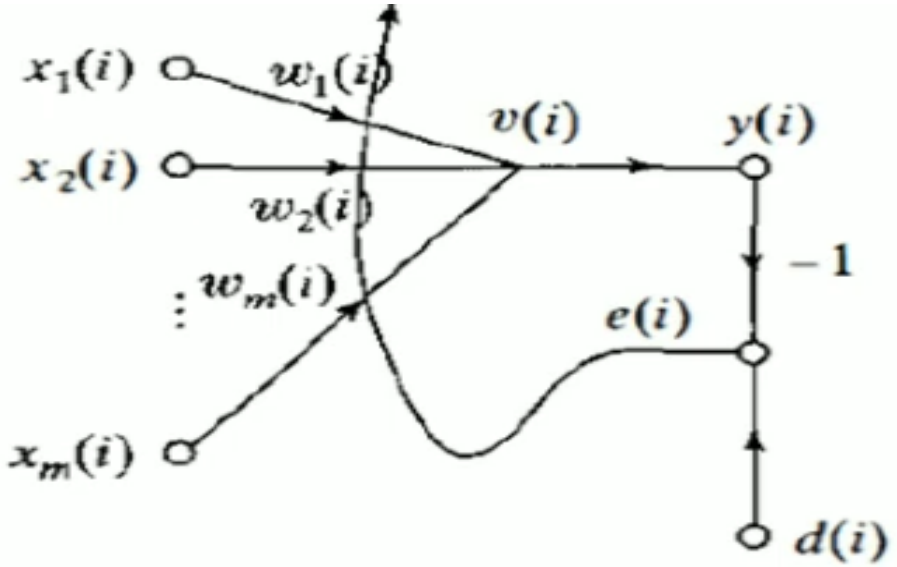
How to design a multiple input-single output model of the unknown dynamical system by building it around a single linear neuron.

The neuronal model operates under the influence of an algorithm that controls necessary adjustments to the synaptic weights of the neuron

- The algorithm starts from an arbitrary setting of the neuron's synaptic weights.

- Adjustments to the synaptic weights. in response to statistical variations in the system's behaviour, are made on a continuous basis.

- Computations of adjustments to the synaptic weights are completed inside a time interval that is one sampling period long.



Its operation consists of two continuous processes:

Project Title

1. Filtering process

2. Adaptive process

Since the neuron is linear, the output y(i) is exactly the same as the induced local field v(i); that is,

$$y(i) = v(i) = \sum_{k=1}^{m} w_k(i)x_k(i)$$

$$y(i) = \mathbf{x}^T(i)\mathbf{w}(i)$$

$$\mathbf{w}(i) = [w_1(i), w_2(i), \ldots, w_m(i)]^T$$

1. *Filtering process*, which involves the computation of two signals:
   - An output, denoted by $y(i)$, that is produced in response to the m elements of the stimulus vector $\mathbf{x}(i)$, namely, $x_1(i), x_2(i), \ldots, x_m(i)$.
   - An error signal, denoted by $e(i)$, that is obtained by comparing the output $y(i)$ to the corresponding output $d(i)$ produced by the unknown system. In effect, $d(i)$ acts as a *desired response* or *target signal*.
2. *Adaptive process*, which involves the automatic adjustment of the synaptic weights of the neuron in accordance with the error signal $e(i)$.